Introduction

If you've ever copy-pasted the same five lines of code into three different places in your script, you already understand the problem functions solve — you just haven't given it a name yet. Functions are the first real lever a beginner pulls to write less code that does more, and they're also one of the first things interviewers probe, because how you write a function reveals how you think: do you name things clearly, do you understand what a function actually gives back to its caller, do you know why `return` and `print` are not the same thing?

This is also where a huge number of beginner bugs are born. Almost every new Python developer eventually writes a function, calls it, tries to use its result, and gets `None` back for reasons they don't understand. Once you understand *why* that happens, you've crossed a real threshold in how you think about code — and that's exactly what this lesson is about.

Problem Overview

In plain terms: a function is a named block of instructions that does nothing on its own — it only runs when you explicitly call it. A **parameter** is a placeholder for input the function expects; an **argument** is the actual value you hand over when calling it. A **return value** is the result the function hands back to whoever called it, so that result can be stored, reused, or passed elsewhere.

On top of these basics, Python gives you a few tools to make functions more flexible:

- **Default arguments** — a fallback value a parameter uses if the caller doesn't supply one.
- **Keyword arguments** — explicitly naming which parameter a value belongs to, instead of relying on position.
- `*args` / `**kwargs` — ways to accept an unknown number of positional or named inputs.

And under all of this sits one quiet rule that trips up almost everyone at least once: if a function never uses `return`, Python hands back `None` by default — silently, with no error.

Example

```
def add(a, b):
    print(a + b)
```

```
result = add(3, 5)   # prints 8
print(result)         # prints None
```

`add` prints `8` to the screen — that part looks correct. But `result` is `None`, because `add` never actually returned anything; it only printed. Printing shows a human something on screen. Returning hands a value back to the program itself. They are unrelated actions that happen to look similar when you're staring at terminal output.

Compare that to a version that returns properly:

```
def add(a, b):
    return a + b

result = add(3, 5)
print(result)   # prints 8
```

Now `result` actually holds `8`, because the function handed that value back to the caller instead of just displaying it.

Intuition

Think of a function like a vending machine. You put in money (arguments), the machine does its internal work (the function body), and it either **hands you a snack** (`return`) or it just **lights up a display** (`print`) without giving you anything you can carry away. If you walk off expecting a snack from a machine that only lights up, your hands are empty — that's exactly what happens when you try to use the result of a function that only prints.

An experienced engineer doesn't discover this through memorization — they discover it the first time a variable that "should" hold a number turns out to hold `None`, and they trace it back to a missing `return`. From then on, the question "does this function return something, or does it just do something?" becomes automatic before writing a single call site.

The same instinct applies to defaults and keyword arguments: a good engineer notices when a function is being called the same way 90% of the time with only one value changing, and reaches for a default. They notice when a function call is hard to read because of unlabeled values in a row, and reach for keyword arguments to label them.

Brute Force Solution

There isn't really a "brute force vs optimal" split for functions themselves — but there is one for *how you avoid repeating logic*, which is the actual problem functions solve.

Idea: copy-paste the same block of code everywhere you need it.

Algorithm: find where you need the logic, paste it, adjust variable names if needed.

Advantages: zero setup, works immediately, no need to think about parameters or naming.

Disadvantages: any bug fix or behavior change means hunting down every copy and fixing them all — and missing even one reintroduces the bug. Readability suffers because the "story" of your program gets buried in repeated implementation detail.

Time/space complexity: not applicable in the algorithmic sense — but "maintenance complexity" scales with the number of copies, which is exactly what functions eliminate.

Optimal Solution

The optimal approach is to identify the repeated logic, wrap it in a function once, and call that function everywhere you need the behavior.

Piece	Purpose
`def name(params):`	Declares the function and what input it accepts
Function body (indented)	The actual work performed
`return value`	Sends a result back to the caller (optional)
Default arguments	Fallback values so common calls stay short
Keyword arguments	Explicit labeling of which value goes where

The key structural rule: **positional arguments must come before keyword arguments** in a call. Python resolves positional arguments by order first, then matches keyword arguments by name, so mixing them in the wrong order raises an error.

Python Code

```
def make_coffee(item, size="medium", milk=True):
    """Build and return a description of a coffee order."""
    milk_text = "with milk" if milk else "without milk"
    return f"{size} {item}, {milk_text}"


def total_of(*nums):
    """Sum an arbitrary number of positional values."""
    return sum(nums)
```

```
def profile(**info):
    """Collect arbitrary keyword info into a dict."""
    return info


def safe_add_item(item, cart=None):
    """Append item to cart, avoiding the mutable default trap."""
    if cart is None:
        cart = []
    cart.append(item)
    return cart


if __name__ == "__main__":
    assert make_coffee("latte") == "medium latte, with milk"
    assert make_coffee("latte", size="large", milk=False) == "large latte, without milk"
    assert total_of(1, 2, 3, 4) == 10
    assert profile(name="Maya", age=30) == {"name": "Maya", "age": 30}
    assert safe_add_item("apple") == ["apple"]
    assert safe_add_item("banana") == ["banana"]  # not ["apple", "banana"]
    print("all checks passed")
```

Code Walkthrough

- `def make_coffee(item, size="medium", milk=True):` — `item` is required, `size` and `milk` have defaults, so callers only need to override what they care about.
- The docstring under `def` documents intent for anyone reading the function later, including future you.
- `return f"..."` sends the built string back to the caller instead of just printing it, so the result can be stored, compared, or passed along.
- `def total_of(*nums):` — the single star gathers any number of positional arguments into a tuple called `nums`, so `total_of(1, 2, 3, 4)` works regardless of how many values are passed.
- `def profile(**info):` — double star gathers named arguments into a dictionary, letting the caller pass any number of labeled values.
- `def safe_add_item(item, cart=None):` — this is the fix for the mutable default trap. `cart` defaults to `None` (immutable, safe), and a fresh list is only created inside the function body when needed.

Dry Run

Call `safe_add_item("banana")` right after `safe_add_item("apple")`:

1. First call: `cart` is `None` → `cart` becomes a brand-new empty list `[]` → `"apple"` is appended → returns `["apple"]`.
2. Second call: `cart` is again `None` (a fresh default check every time) → a brand-new empty list is created again → `"banana"` is appended → returns `["banana"]`.

If `cart=[]` had been written directly in the signature instead, Python would create that list **once**, at function definition time, and reuse the same list object on every call — so the second call would return `["apple", "banana"]`, which is almost never what you want.

Complexity Analysis

- **Time:** each of these functions does O(1) work aside from `total_of`, which is O(n) in the number of arguments passed (it has to sum them all) — correct because summing n values fundamentally requires touching each one.
- **Space:** O(1) extra space per call for the simple functions; O(n) for `total_of` and `profile` since `*args` / `**kwargs` must materialize a tuple/dict to hold the collected values — this is unavoidable since Python needs somewhere to store an unknown number of inputs.

Alternative Solutions

Instead of `*args`, you could require callers to pass a single list: `total_of([1, 2, 3, 4])`. Functionally equivalent — the difference is purely how the caller experiences it. `*args` reads more naturally for "call this with however many numbers you want," while an explicit list parameter is clearer when the caller already has a collection built elsewhere. Neither is objectively better; it depends on how you expect the function to be called.

Similarly, a `lambda` could replace a tiny one-expression function (`square = lambda n: n * n`), but a regular `def` function is almost always more readable once anything beyond a single trivial expression is involved.

Edge Cases

- **No arguments passed to a function with all defaults:** should fall back cleanly to every default value.
- **Mutable default argument (list/dict):** must use `None` + create-inside-body pattern, or state shared across calls silently corrupts results.

- **Misspelled keyword argument:** raises `TypeError: unexpected keyword argument`, since Python matches keyword arguments by exact name.
- **Function with no `return` statement:** returns `None`, not an error — a valid but easily-forgotten case.
- **Mixing positional and keyword arguments in the wrong order:** positional arguments must come first, or Python raises a `SyntaxError` / `TypeError`.

Common Mistakes

1. Calling a function without parentheses (`greet` instead of `greet()`) — this references the function object rather than running it.
2. Confusing `print` with `return` — the function *looks* like it works because output appears on screen, but the caller gets `None` back.
3. Using a mutable object (`[]`, `{}`) as a default argument, causing state to leak between unrelated calls.
4. Misspelling a keyword argument name, causing a `TypeError`.
5. Assuming a parameter change inside a function affects the original variable outside it — for simple types like numbers and strings, it does not, since Python passes the value, not the original variable.
6. Shadowing a built-in function name (like `sum`) with your own function of the same name, silently breaking any code that expected the original.

Interview Questions

- What's the difference between a parameter and an argument?
- Why does a function with no `return` statement return `None` instead of raising an error?
- Explain the mutable default argument bug and how you'd fix it.
- What's the difference between `*args` and `**kwargs`?
- Can positional and keyword arguments be mixed in one call? What's the rule?
- What is variable scope, and why does a variable defined inside a function disappear outside it?

Similar Problems

- **LeetCode-style "implement a function with default behavior"** problems — test the same default-argument reasoning shown here.
- **Two Sum** and similar array problems — a great next step for practicing writing a function with clear parameters and a single well-defined return value.

- **Recursive problems (factorial, Fibonacci)** — build directly on the function-calling-a-function idea covered here.

Key Takeaways

A function is reusable, named logic that only runs when called. Parameters wait, arguments arrive. `return` gives your program something to actually use — `print` only shows a human something on screen. Default arguments keep common calls short; keyword arguments make calls with many parameters readable and unambiguous. And the single most common beginner bug — a mutable default argument — is avoided by defaulting to `None` and creating the mutable object fresh inside the function body.

Watch the Video

For the full walkthrough with live coding and every example explained step by step, watch the video here: {LINK}

About the Series

This lesson is part of the Daily Python LeetCode series — a day-by-day walkthrough of core Python fundamentals and interview-style problems, built for anyone going from "I can read Python" to "I can confidently write and reason about it under pressure."

Call To Action

If this cleared up something that's been quietly confusing you, do three things: like the video on YouTube so it reaches more people learning the same thing, subscribe to the channel so you don't miss the next lesson, and subscribe to the Substack for the written version of every lesson delivered straight to your inbox. Drop a comment with the trickiest function bug you've personally run into — and share this with anyone still fighting the "why is this None" bug.

Quick Review

Why write a function instead of copy-pasting code?

One change updates every call site; copy-pasted code drifts and bugs get fixed in only one spot.

Function vs parameter vs argument — what's the difference?

Function: named reusable block. Parameter: placeholder name in def. Argument: actual value passed when calling.

What does 'return value' let a function do?

Send a result back to the caller so it can be stored, printed, or used in another expression.

Default argument vs keyword argument?

Default: value used if caller omits it (def f(x=5)). Keyword: caller names the param when calling (f(x=5)).

Why does parameter order matter in positional calls?

Positional args fill parameters left to right by position; swapping order sends values to the wrong parameter.

Common bug: function has no return statement?

It returns None implicitly; using the result elsewhere silently produces None instead of the intended value.
